

TCS-504 – SYSTEM DESIGN

ASSIGNMENT 1

MOVIE TICKET BOOKING SYSTEM

Submitted By :

Name : Rudransh

Roll No. : 48

Section : B2

Course : B.tech. CSE

Semester : 5

Subject : System Design

Subject Code : TCS-504

1. Requirement Analysis

1.1 Functional Requirements

FR1 — List Movies

The system shall display all movies currently playing in the cinema, including their title, language and duration.

FR2 — List Shows

For a selected movie, the system shall display all available shows with their screen number and start time.

FR3 — Display Seat Availability

For a selected show, the system shall display the seat layout and show each seat as either AVAILABLE or BOOKED.

FR4 — Booking

A customer selects one or more seat numbers for a show. If any selected seat is already BOOKED, the whole booking is rejected and no seat changes state. Booking is confirmed only after payment succeeds.

FR5 — Calculate Booking Price

The system shall calculate the total booking amount according to seat type: SILVER ₹150, GOLD ₹250 and PLATINUM ₹400.

FR6 — Payment

Exactly one payment method (UPI / Card / Cash) shall be used per booking. If payment fails, seats are released and booking status becomes FAILED.

FR7 — Print Ticket

After successful payment, the system shall print a ticket containing booking ID, movie, screen, show time, selected seat numbers and total amount.

FR8 — Cancel Booking

The system shall allow a customer to cancel a booking. After cancellation, the seats associated with that booking shall become AVAILABLE again.

1.2 Non-Functional Requirements

NFR1 — Modularity

The system shall be divided into separate classes so that each class has a clear responsibility.

NFR2 — Extensibility

A new payment method should be addable without requiring major changes to the existing booking logic.

NFR3 — Input Validation

Invalid menu choices and invalid seat numbers shall be handled with a clear error message without crashing the program.

NFR4 — Maintainability

The system shall use meaningful names, avoid repeated code and follow basic SOLID principles.

2. NOUN-VERB ANALYSIS

2.1 NOUN ANALYSIS

The following nouns were identified from the problem statement and converted into candidate classes or attributes.

1. Movie
Class: Yes
Reason: Represents movie information such as title, language and duration.
2. Seat
Class: Yes
Reason: Represents a physical seat in a screen.
3. Screen
Class: Yes
Reason: Represents a screen containing seats and hosting shows.
4. Cinema
Class: Yes
Reason: Represents the cinema and manages its screens.
5. Show
Class: Yes
Reason: Represents a particular movie show with a start time and screen.
6. ShowSeat
Class: Yes
Reason: Maintains the booking status of a seat for a particular show.
7. Customer
Class: Yes
Reason: Represents the customer who makes a booking.
8. Booking
Class: Yes
Reason: Stores information about a customer's seat booking.
9. Payment
Class: Yes
Reason: Represents the payment operation for a booking.
10. UpiPayment
Class: Yes
Reason: Represents UPI as a specific payment method.
11. CardPayment
Class: Yes
Reason: Represents Card as a specific payment method.
12. CashPayment
Class: Yes
Reason: Represents Cash as a specific payment method.
13. Price Calculator
Class: Yes
Reason: Calculates the total price according to seat types.

- 14. Ticket Printer
Class: Yes
Reason: Prints the ticket after successful payment.
- 15. Booking Service
Class: Yes
Reason: Handles the main booking operations.
- 16. Seat Layout
Class: No
Reason: It is only a representation of seat information.
- 17. Movie Title
Class: No
Reason: It is an attribute of the Movie class.
- 18. Start Time
Class: No
Reason: It is an attribute of the Show class.
- 19. Booking ID
Class: No
Reason: It is an attribute of the Booking class.

2.2 VERB ANALYSIS

The following verbs represent the main behaviours or operations of the system.

- 1. Display movies
Method: displayMovies()
- 2. Display shows
Method: displayShows()
- 3. Display seats
Method: displaySeats()
- 4. Select seats
Method: selectSeats()
- 5. Check availability
Method: checkAvailability()
- 6. Calculate price
Method: calculatePrice()
- 7. Make payment
Method: pay()
- 8. Confirm booking
Method: confirmBooking()
- 9. Cancel booking
Method: cancelBooking()
- 10. Release seats
Method: releaseSeats()
- 11. Print ticket
Method: printTicket()

3. CLASS DESIGN

The following classes are identified for the Movie Ticket Booking System. Each class has a specific responsibility, attributes and methods.

3.1 Movie

Responsibility:

Stores information about a movie.

Attributes:

- title
- language
- duration

Methods:

- getTitle()
- getLanguage()
- getDuration()
- displayMovie()

3.2 Seat

Responsibility:

Represents a physical seat in a screen.

Attributes:

- seatNumber
- seatType

Methods:

- getSeatNumber()
- getSeatType()
- getPrice()

3.3 Screen

Responsibility:

Represents a cinema screen and its physical seats.

Attributes:

- screenNumber
- seats

Methods:

- addSeat()
- getSeats()
- displaySeats()

3.4 Cinema

Responsibility:

Manages the screens available in the cinema.

Attributes:

- name
- screens

Methods:

- addScreen()
- getScreens()
- displayScreens()

3.5 Show

Responsibility:

Represents a particular movie show.

Attributes:

- movie
- screen
- startTime

Methods:

- getMovie()
- getScreen()
- getStartTime()
- displayShow()

3.6 ShowSeat

Responsibility:

Maintains the availability status of a physical seat for a particular show.

Attributes:

- seat
- status

Methods:

- isAvailable()
- book()
- release()

3.7 Customer

Responsibility:

Stores customer information.

Attributes:

- customerId
- name
- phone

Methods:

- getCustomerId()
- getName()
- getPhone()

3.8 Booking

Responsibility:

Stores the details and status of a customer's booking.

Attributes:

- bookingId
- customer
- show

- selectedSeats
 - totalAmount
 - status
- Methods:
- confirm()
 - cancel()
 - getTotalAmount()
 - getStatus()

3.9 Payment

Responsibility:

Defines the common payment behaviour.

Attributes:

- amount

Methods:

- pay()

3.10 UpiPayment

Responsibility:

Processes payment using UPI.

Methods:

- pay()

3.11 CardPayment

Responsibility:

Processes payment using Card.

Methods:

- pay()

3.12 CashPayment

Responsibility:

Processes payment using Cash.

Methods:

- pay()

3.13 PriceCalculator

Responsibility:

Calculates the total booking price according to seat type.

Methods:

- calculatePrice()

3.14 TicketPrinter

Responsibility:

Prints the ticket after successful payment.

Methods:

- printTicket()

3.15 BookingService

Responsibility:

Handles the main booking operations.

Methods:

- checkAvailability()
- createBooking()
- confirmBooking()
- cancelBooking()
- releaseSeats()

4. RELATIONSHIP ANALYSIS

The following relationships are used between the classes in the Movie Ticket Booking System.

4.1 Cinema — Screen

Relationship: Composition

Justification:

A Cinema owns its Screens. A Screen is considered a part of the Cinema in this system and is managed by the Cinema.

4.2 Screen — Seat

Relationship: Composition

Justification:

A Screen contains its physical Seats. The Seats belong to the Screen and are managed as part of its seat layout.

4.3 Show — Movie

Relationship: Aggregation

Justification:

A Show uses a Movie, but the Movie can exist independently of a particular Show. Therefore, the Movie is not destroyed when a Show ends or is cancelled.

4.4 Show — Screen

Relationship: Association

Justification:

A Show is scheduled on a particular Screen. The Screen exists independently and can host different Shows at different times.

4.5 Show — ShowSeat

Relationship: Composition

Justification:

ShowSeat maintains the booking status of a physical Seat specifically for a particular Show. Therefore, it belongs to that Show.

4.6 ShowSeat — Seat

Relationship: Association

Justification:

ShowSeat refers to a physical Seat and maintains its status for a particular Show. The physical Seat itself can exist independently.

4.7 Booking — Customer

Relationship: Association

Justification:

A Booking is made by a Customer. Both the Customer and Booking can exist independently.

4.8 Booking — Show

Relationship: Association

Justification:

A Booking is associated with a particular Show selected by the Customer.

4.9 Booking — ShowSeat

Relationship: Association

Justification:

A Booking contains the ShowSeats selected by the Customer for that particular Show.

4.10 Payment — UpiPayment

Relationship: Inheritance

Justification:

UpiPayment is a specific type of Payment and implements the common payment behaviour defined by Payment.

4.11 Payment — CardPayment

Relationship: Inheritance

Justification:

CardPayment is a specific type of Payment and implements the common payment behaviour defined by Payment.

4.12 Payment — CashPayment

Relationship: Inheritance

Justification:

CashPayment is a specific type of Payment and implements the common payment behaviour defined by Payment.

4.13 BookingService — Booking

Relationship: Association

Justification:

BookingService manages the creation, confirmation and cancellation of Booking objects.

4.14 BookingService — PriceCalculator

Relationship: Association

Justification:

BookingService uses PriceCalculator to calculate the total amount for the selected seats.

4.15 BookingService — TicketPrinter

Relationship: Association

Justification:

BookingService uses TicketPrinter to print the ticket after successful payment.

5. CLASS DIAGRAM

The following class diagram represents the structure of the Movie Ticket Booking System. It shows the classes, their attributes, methods, relationships, multiplicities, and the Payment inheritance hierarchy.

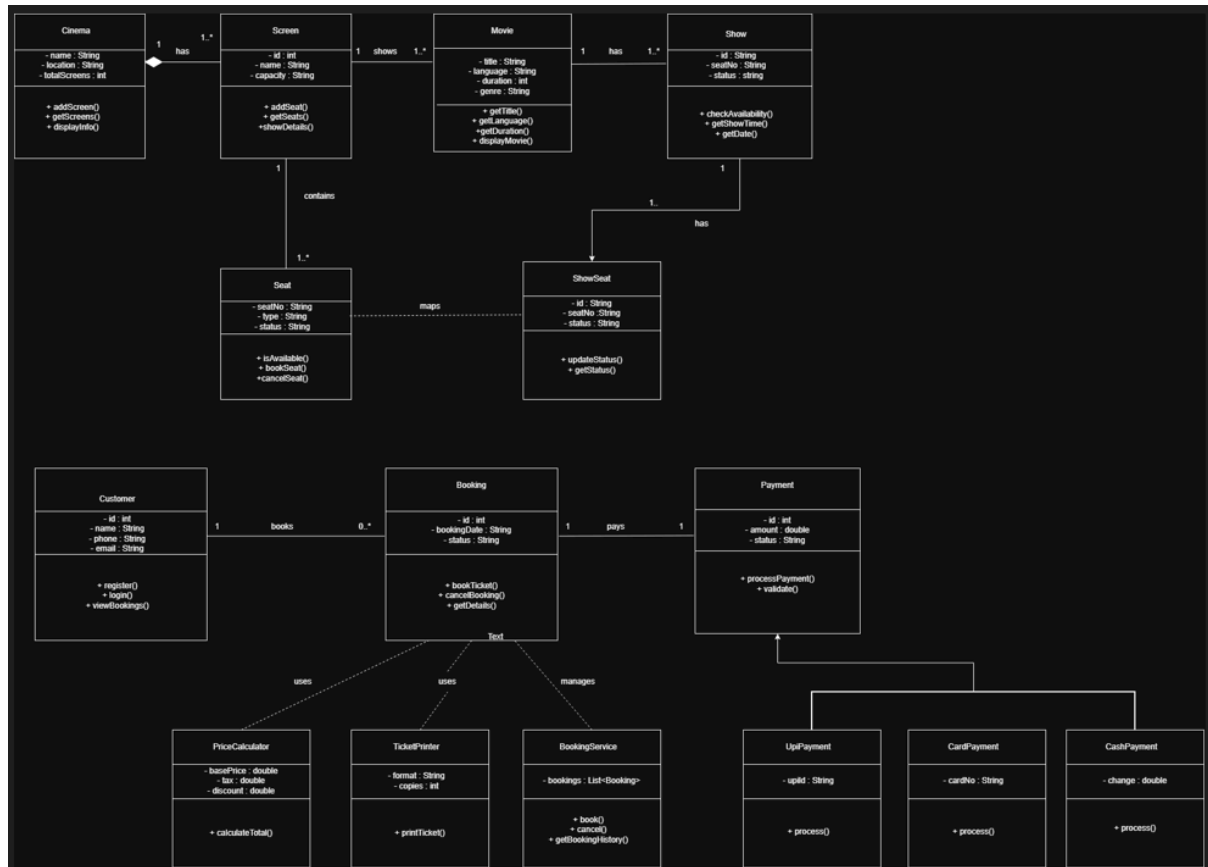


Figure 1: Class Diagram of Movie Ticket Booking System

6. SEQUENCE DIAGRAM

Use Case: Customer Books 1 Seat and Pays by UPI

The sequence diagram shows the interaction between the customer and the system when a customer selects a show, chooses one available seat, calculates the price, makes payment through UPI, confirms the booking, and receives the printed ticket.

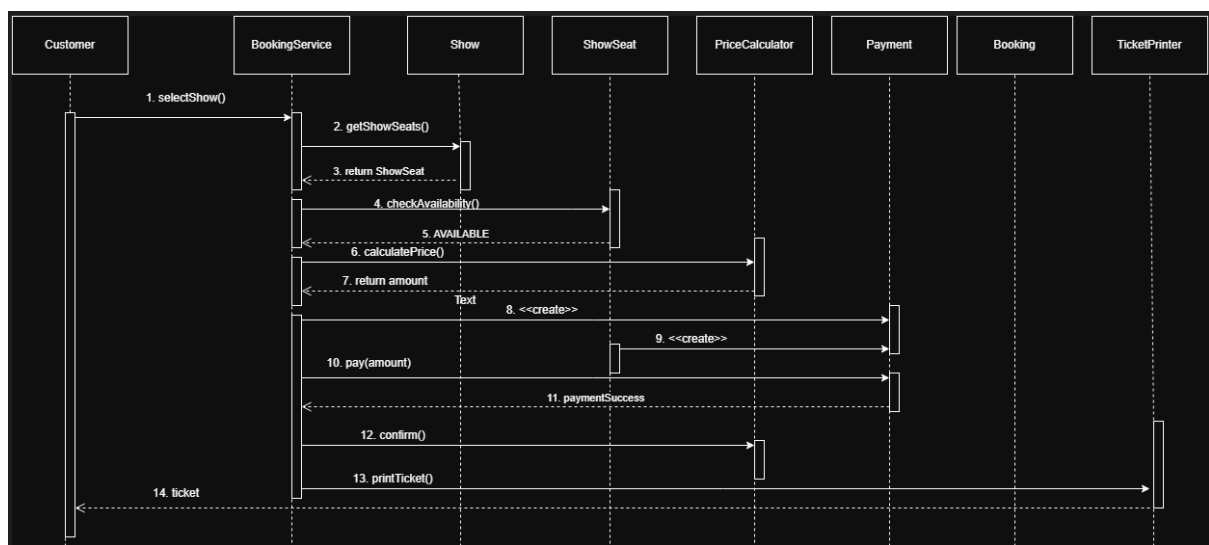


Figure 2: Sequence Diagram for Customer Booking and UPI Payment

7. Modular Working C++ Code + Demo

The Movie Ticket Booking System is implemented in C++ using a modular class-based approach. Each major class is implemented in a separate .cpp file. The system supports movie selection, seat booking, multiple seat selection, payment, ticket printing and cancellation.

7.1 Movie.cpp

```
#include<iostream>
#include<string>
using namespace std;
class Movie{
    private:
        string title;
        string language;
        int duration;
    public:
        Movie(string title,string language,int duration) :
        title(title),language(language),duration(duration){}
        string getTitle(){
            return title;
        }
        string getLanguage(){
            return language;
        }
        int getDuration(){
            return duration;
        }
        void showMovie(){
            cout<<title<<" "<<language<<"   "<<duration<<" min"<<endl;
        }
};
```

7.2 Seat.cpp

```
#include<iostream>
#include<string>
using namespace std;
class Seat{
    private:
        string number;
        string type;
    public:
        Seat(string number,string type) : number(number),type(type){}
```

```

    string getNumber(){
        return number;
    }
    string getType(){
        return type;
    }
    int getPrice(){
        if(type=="SILVER")
            return 150;
        else if(type=="GOLD")
            return 250;
        else
            return 400;
    }
    void showSeat(){
        cout<<number<<" "<<type<<" Rs. "<<getPrice()<<endl;
    }
}

```

//

7.3 Screen.cpp

```

#include <iostream>
#include <vector>
using namespace std;
class Screen {
private:
    int screenNumber;
    vector<Seat> seats;
public:
    Screen(int number) : screenNumber(number) {
        seats.push_back(Seat("A1", "SILVER"));
        seats.push_back(Seat("A2", "SILVER"));
        seats.push_back(Seat("B1", "GOLD"));
        seats.push_back(Seat("B2", "GOLD"));
        seats.push_back(Seat("C1", "PLATINUM"));
        seats.push_back(Seat("C2", "PLATINUM"));
    }
    void showSeats() {
        cout << "\nScreen " << screenNumber << " Seats:\n";
        for (Seat &seat : seats)
            seat.showSeat();
    }
    int getScreenNumber() {

```

```

        return screenNumber;
    }
};

```

7.4 Cinema.cpp

```

#include<iostream>
#include<vector>
#include<string>
using namespace std;
class Cinema{
    private:
        string name;
        vector<int> screens;
    public:
        Cinema(string name) : name(name) {
            screens.push_back(1);
            screens.push_back(2);
        }
        void addScreen(int screenNumber) {
            screens.push_back(screenNumber);
        }
        void showScreens() {
            cout<<"\nCinema: "<<name<<endl;
            cout<<"Available Screens: ";
            for(int screen : screens)
                cout<<screen<<" ";
            cout<<endl;
        }
        string getName() {
            return name;
        }
};

```

7.5 Show.cpp

```

#include<iostream>
#include<string>
using namespace std;
class Show{
    private:
        int showId;

```

```

    string movieName;
    string time;
    int screenNumber;
public:
    Show(int id,string movie,string showTime,int screen) :
showId(id),movieName(movie),time(showTime),screenNumber(screen){}
    int getShowId(){
        return showId;
    }
    string getMovieName(){
        return movieName;
    }
    string getTime(){
        return time;
    }
    int getScreenNumber(){
        return screenNumber;
    }
    void showDetails(){
        cout<<"Show ID: "<<showId<<endl;
        cout<<"Movie: "<<movieName<<endl;
        cout<<"Time: "<<time<<endl;
        cout<<"Screen: "<<screenNumber<<endl;
    }
};

```

7.6 ShowSeat.cpp

```

#include<iostream>
#include<string>
using namespace std;
class ShowSeat{
private:
    string seatNumber;
    string seatType;
    bool booked;
public:
    ShowSeat(string number,string type) :
seatNumber(number),seatType(type),booked(false){}
    string getSeatNumber(){
        return seatNumber;
    }
};

```

```

    string getSeatType(){
        return seatType;
    }
    bool isBooked(){
        return booked;
    }
    int getPrice(){
        if(seatType=="SILVER")
            return 150;
        else if(seatType=="GOLD")
            return 250;
        else
            return 400;
    }
    bool bookSeat(){
        if(booked){
            cout<<"Seat "<<seatNumber<<" is already booked!"<<endl;
            return false;
        }
        booked=true;
        return true;
    }
    void cancelSeat(){
        booked=false;
    }
    void showSeat(){
        cout<<seatNumber<<" | "<<seatType<<" | Rs."<<getPrice()<<" | 
";

        if(booked)
            cout<<"BOOKED";
        else
            cout<<"AVAILABLE";
        cout<<endl;
    }
}

```

7.7 Customer.cpp

```

#include<iostream>
#include<string>
using namespace std;
class Customer{

```

```

    private:
        int customerId;
        string name;
        string email;
    public:
    Customer(int id, string name, string email) {
        this->customerId = id;
        this->name = name;
        this->email = email;
    }

    int getCustomerId() {
        return customerId;
    }

    string getName() {
        return name;
    }

    string getEmail() {
        return email;
    }

    void showCustomer() {
        cout<<"\nCustomer ID: "<<customerId<<endl;
        cout<<"Name: "<<name<<endl;
        cout<<"Email: "<<email<<endl;
    }
};

```

7.8 Booking.cpp

```

#include <iostream>
#include <vector>
#include <string>
using namespace std;
class Booking {
private:
    static int nextBookingId;
    int bookingId;
    int customerId;
    string movieName;
    vector<string> seats;
    int totalAmount;
    bool confirmed;

```

```

public:
    Booking(int customer, string movie)
        : bookingId(++nextBookingId),
          customerId(customer),
          movieName(movie),
          totalAmount(0),
          confirmed(false) {}
    void addSeat(string seatNumber, int price) {
        seats.push_back(seatNumber);
        totalAmount += price;
    }
    void confirmBooking() {
        confirmed = true;
    }
    void cancelBooking()
    { confirmed =
      false;
    }
    bool isConfirmed() {
        return confirmed;
    }
    int getBookingId() {
        return bookingId;
    }
    int getTotalAmount() {
        return totalAmount;
    }
    void showBooking() {
        cout << "\nBooking ID: " << bookingId <<
        endl; cout << "Customer ID: " << customerId
        << endl; cout << "Movie: " << movieName <<
        endl;
        cout << "Seats: ";
        for (string seat :
            seats) cout << seat
            << " ";
        cout << "\nTotal Amount: Rs." << totalAmount <<
        endl; if (confirmed)
            cout << "Status: CONFIRMED" << endl;
        else
            cout << "Status: CANCELLED" << endl;
    }
};

int Booking::nextBookingId = 1000;

```

7.9 Payment.cpp

```
#include<iostream>
#include<string>
using namespace std;
class Payment{
protected:
    int amount;
    bool status;
public:
    Payment(int amount) : amount(amount),status(false){}
    virtual bool makePayment()=0;
    bool isSuccessful(){
        return status;
    }
    int getAmount(){
        return amount;
    }
};
```

7.10 PaymentTypes.cpp

```
#include <iostream>
using namespace std;
class UpiPayment : public Payment {
public:
    UpiPayment(int amount) : Payment(amount) {}
    bool makePayment() override {
        cout << "UPI Payment of Rs." << amount << " successful." <<
endl;
        status = true;
        return status;
    }
};
class CardPayment : public Payment {
public:
    CardPayment(int amount) : Payment(amount) {}
    bool makePayment() override {
        cout << "Card Payment of Rs." << amount << " successful." <<
endl;
        status = true;
        return status;
    }
};
```

```

);
class CashPayment : public Payment {
public:
    CashPayment(int amount) : Payment(amount) {}
    bool makePayment() override {
        cout << "Cash Payment of Rs." << amount << " successful." <<
endl;
        status = true;
        return status;
    }
};

```

7.11 PriceCalculator.cpp

```

#include <iostream>
#include <vector>
using namespace std;
class PriceCalculator {
public:
    int calculatePrice(vector<int> prices) {
        int total = 0;
        for (int price : prices)
            total += price;
        return total;
    }
    int calculatePrice(int price, int quantity) {
        return price * quantity;
    }
};

```

7.12 TicketPrinter.cpp

```

#include <iostream>
#include <string>
#include <vector>
using namespace std;
class TicketPrinter {
public:
    void printTicket(int bookingId, string customerName,
                    string movieName, vector<string> seats,
                    int totalAmount) {
        cout << "\n===== MOVIE TICKET =====\n";
    }
};

```

```

        cout << "Booking ID: " << bookingId << endl;
        cout << "Customer: " << customerName << endl;
        cout << "Movie: " << movieName << endl;
        cout << "Seats: ";
        for (string seat : seats)
            cout << seat << " ";
        cout << "\nTotal Amount: Rs." << totalAmount << endl;
        cout << "Status: CONFIRMED" << endl;
        cout << "=====\n";
    }
}

```

7.13 BookingService.cpp

```

#include <iostream>
#include <vector>
#include <string>
using namespace std;
class BookingService {
public:
    bool bookSeats(vector<ShowSeat>& seats, vector<string>
selectedSeats) {
        for (string selected : selectedSeats) {
            bool found = false;
            for (ShowSeat& seat : seats) {
                if (seat.getSeatNumber() == selected) {
                    found = true;
                    if (seat.isBooked()) {
                        cout << "Seat " << selected << " is already
booked!" << endl;
                        return false;
                    }
                    break;
                }
            }
            if (!found) {
                cout << "Invalid seat: " << selected << endl;
                return false;
            }
        }
        for (string selected : selectedSeats) {
            for (ShowSeat& seat : seats) {

```

```

        if (seat.getSeatNumber() == selected) {
            seat.bookSeat();
            break;
        }
    }
}
cout << "Seats booked successfully." << endl;
return true;
}

void cancelSeats(vector<ShowSeat>& seats, vector<string>
selectedSeats) {
    for (string selected : selectedSeats) {
        for (ShowSeat& seat : seats) {
            if (seat.getSeatNumber() == selected) {
                seat.cancelSeat();
                break;
            }
        }
    }
    cout << "Booking cancelled. Seats are available again." <<
endl;
}
};

```

7.14 main.cpp

```

#include "Movie.cpp"
#include "Seat.cpp"
#include "Screen.cpp"
#include "Cinema.cpp"
#include "Show.cpp"
#include "ShowSeat.cpp"
#include "Customer.cpp"
#include "Booking.cpp"
#include "Payment.cpp"
#include "PaymentTypes.cpp"
#include "PriceCalculator.cpp"
#include "TicketPrinter.cpp"
#include "BookingService.cpp"
int main() {
    Movie movie("Avengers", "English", 180);
    Customer customer(101, "Raghav", "raghav@gmail.com");
}

```

```

Show show(1, movie.getTitle(), "7:00 PM", 1);
vector<ShowSeat> seats = {
    ShowSeat("A1", "SILVER"),
    ShowSeat("A2", "SILVER"),
    ShowSeat("B1", "GOLD"),
    ShowSeat("B2", "GOLD"),
    ShowSeat("C1", "PLATINUM"),
    ShowSeat("C2", "PLATINUM")
};
BookingService service;
while (true) {
    cout << "\n===== MOVIE TICKET BOOKING SYSTEM =====\n";
    cout << "1. Show Movie\n";
    cout << "2. Show Seats\n";
    cout << "3. Book Seats\n";
    cout << "4. Cancel Seats\n";
    cout << "0. Exit\n";
    cout << "Enter choice: ";
    int choice;
    cin >> choice;
    if (choice == 0) {
        cout << "Thank you!\n";
        break;
    }
    if (choice == 1) {
        movie.showMovie();
        show.showDetails();
    }
    else if (choice == 2) {
        for (ShowSeat &seat : seats)
            seat.showSeat();
    }
    else if (choice == 3) {
        string s1, s2;
        cout << "Enter first seat: ";
        cin >> s1;
        cout << "Enter second seat: ";
        cin >> s2;
        vector<string> selected = {s1, s2};
        if (service.bookSeats(seats, selected)) {
            int total = 0;
            for (ShowSeat &seat : seats) {
                for (string s : selected) {

```

```

        if (seat.getSeatNumber() == s)
            total += seat.getPrice();
    }
}

cout << "Total Amount: Rs." << total << endl;
cout << "Payment Method:\n";
cout << "1. UPI\n";
cout << "2.
Card\n"; cout <<
"3. Cash\n";
cout << "Enter choice: ";

int paymentChoice;
cin >> paymentChoice;
Payment *payment = nullptr;
if (paymentChoice == 1)
    payment = new UpiPayment(total);
else if (paymentChoice == 2)
    payment = new CardPayment(total);
else if (paymentChoice == 3)
    payment = new CashPayment(total);
else if (paymentChoice == 4) {
    cout << "Payment failed!" << endl;
    service.cancelSeats(seats, selected);
    cout << "Seats released successfully." << endl;
    continue;
}
else {
    cout << "Invalid payment method." << endl;
    service.cancelSeats(seats, selected);
    continue;
}

if (payment->makePayment()) {
    Booking booking(customer.getCustomerId(),
movie.getTitle());
    for (string s : selected) {
        for (ShowSeat &seat : seats) {
            if (seat.getSeatNumber() == s)
                booking.addSeat(s, seat.getPrice());
        }
    }
    booking.confirmBooking();
    TicketPrinter printer;
    printer.printTicket(
        booking.getBookingId(),

```

```
        customer.getName(),
        movie.getTitle(),
        selected,
        total

    );

    }

    delete payment;
}

}

else if (choice == 4) {
    string s1, s2;
    cout << "Enter first seat: ";
    cin >> s1;
    cout << "Enter second seat: ";
    cin >> s2;
    vector<string> selected = {s1, s2};
    service.cancelSeats(seats, selected);
}

else {
    cout << "Invalid choice!\n";
}

}

return 0;
```

Demo Output

```
PS C:\Users\gigabyte\OneDrive\Desktop\SYSTEM DESIGN> g++ main.cpp -o main
PS C:\Users\gigabyte\OneDrive\Desktop\SYSTEM DESIGN> ./main
```

```
===== MOVIE TICKET BOOKING SYSTEM =====
1. Show Movie
2. Show Seats
3. Book Seats
4. Cancel Seats
0. Exit
```

```
Enter choice: 1
Avengers  English  180 min
Show ID: 1
Movie: Avengers
Time: 7:00 PM
Screen: 1
```

```
===== MOVIE TICKET BOOKING SYSTEM =====
1. Show Movie
2. Show Seats
3. Book Seats
4. Cancel Seats
0. Exit
```

```
===== MOVIE TICKET BOOKING SYSTEM =====
1. Show Movie
2. Show Seats
3. Book Seats
4. Cancel Seats
0. Exit
Enter choice: 3
Enter first seat: A1
Enter second seat: B1
Seats booked successfully.
Total Amount: Rs.400
Payment Method:
1. UPI
2. Card
3. Cash
```

===== MOVIE TICKET BOOKING SYSTEM =====

1. Show Movie
2. Show Seats
3. Book Seats
4. Cancel Seats
0. Exit

Enter choice: 3

Enter first seat: A1

Enter second seat: B1

Seats booked successfully.

Total Amount: Rs.400

Payment Method:

1. UPI
2. Card
3. Cash

Enter choice: 3

Cash Payment of Rs.400 successful.

===== MOVIE TICKET =====

Booking ID: 1001

Customer: Raghav

Movie: Avengers

Seats: A1 B1

Total Amount: Rs.400

Status: CONFIRMED

=====

===== MOVIE TICKET BOOKING SYSTEM =====

1. Show Movie
2. Show Seats
3. Book Seats
4. Cancel Seats
0. Exit

===== MOVIE TICKET BOOKING SYSTEM =====

1. Show Movie

2. Show Seats

3. Book Seats

4. Cancel Seats

0. Exit

Enter choice: 3

Enter first seat: A1

Enter second seat: B1

Seats booked successfully.

Total Amount: Rs.400

Payment Method:

1. UPI

2. Card

3. Cash

===== MOVIE TICKET BOOKING SYSTEM =====

1. Show Movie

2. Show Seats

3. Book Seats

4. Cancel Seats

0. Exit

Enter choice: 3

Enter first seat: A1

Enter second seat: B1

Seats booked successfully.

Total Amount: Rs.400

Payment Method:

1. UPI

2. Card

3. Cash

Enter choice: 1

UPI Payment of Rs.400 successful.

===== MOVIE TICKET =====

Booking ID: 1001

Customer: Raghav

Movie: Avengers

Seats: A1 B1

Total Amount: Rs.400

Status: CONFIRMED

=====

```
===== MOVIE TICKET BOOKING SYSTEM =====
```

```
1. Show Movie
```

```
2. Show Seats
```

```
3. Book Seats
```

```
4. Cancel Seats
```

```
0. Exit
```

```
Enter choice: 3
```

```
Enter first seat: A1
```

```
Enter second seat: C1
```

```
Seat A1 is already booked!
```

```
===== MOVIE TICKET BOOKING SYSTEM =====
```

```
1. Show Movie
```

```
2. Show Seats
```

```
3. Book Seats
```

```
4. Cancel Seats
```

```
0. Exit
```

```
Enter choice: 3
```

```
Enter first seat: X1
```

```
Enter second seat: A2
```

```
Invalid seat: X1
```

===== MOVIE TICKET BOOKING SYSTEM =====

- 1. Show Movie
- 2. Show Seats
- 3. Book Seats
- 4. Cancel Seats
- 0. Exit

Enter choice: 1

Avengers English 180 min

Show ID: 1

Movie: Avengers

Time: 7:00 PM

Screen: 1

===== MOVIE TICKET BOOKING SYSTEM =====

- 1. Show Movie
- 2. Show Seats
- 3. Book Seats
- 4. Cancel Seats
- 0. Exit

Enter choice: 4

Enter first seat: A1

Enter second seat: B1

Booking cancelled. Seats are available again.

===== MOVIE TICKET BOOKING SYSTEM =====

- 1. Show Movie
- 2. Show Seats
- 3. Book Seats
- 4. Cancel Seats
- 0. Exit

Enter choice: 2

A1 | SILVER | Rs.150 | AVAILABLE

A2 | SILVER | Rs.150 | AVAILABLE

B1 | GOLD | Rs.250 | AVAILABLE

B2 | GOLD | Rs.250 | AVAILABLE

C1 | PLATINUM | Rs.400 | AVAILABLE

C2 | PLATINUM | Rs.400 | AVAILABLE

===== MOVIE TICKET BOOKING SYSTEM =====

1. Show Movie
2. Show Seats
3. Book Seats
4. Cancel Seats
0. Exit

Enter choice: 3

Enter first seat: A1

Enter second seat: B1

Seats booked successfully.

Total Amount: Rs.400

Payment Method:

1. UPI
2. Card
3. Cash

Enter choice: 4

Payment failed!

Booking cancelled. Seats are available again.

Seats released successfully.

===== MOVIE TICKET BOOKING SYSTEM =====

1. Show Movie
2. Show Seats
3. Book Seats
4. Cancel Seats
0. Exit

Enter choice: 2

A1 | SILVER | Rs.150 | AVAILABLE

A2 | SILVER | Rs.150 | AVAILABLE

B1 | GOLD | Rs.250 | AVAILABLE

B2 | GOLD | Rs.250 | AVAILABLE

C1 | PLATINUM | Rs.400 | AVAILABLE

C2 | PLATINUM | Rs.400 | AVAILABLE

8. OOP Concepts Used

The Movie Ticket Booking System uses the following Object-Oriented Programming concepts:

1. Encapsulation

Data members of classes such as Movie, Seat, Customer and Booking are kept private and accessed through public member functions.

2. Abstraction

The Payment class is an abstract class that provides the common structure for different payment methods.

3. Inheritance

UpiPayment, CardPayment and CashPayment classes inherit the Payment class.

4. Runtime Polymorphism

The makePayment() function is overridden by UpiPayment, CardPayment and CashPayment classes and called using the Payment pointer.

5. Compile-time Polymorphism

The PriceCalculator class uses function overloading with different versions of calculatePrice().

6. Static Members

A static member nextBookingId is used in the Booking class to automatically generate booking IDs.

7. this Pointer

This pointer is used in the Customer constructor to refer to the current object's data members.

8. Composition

A Screen contains Seat objects. The seats are created and managed by the Screen class.

9. Aggregation

The Cinema class manages its screens, representing an aggregation relationship.

10. Association

A Customer is associated with a Booking because a customer can make a booking.

9. SOLID Principles

1. Single Responsibility Principle (SRP)

Each class is given a specific responsibility. For example, TicketPrinter is responsible for printing tickets and PriceCalculator is responsible for calculating ticket prices.

2. Open/Closed Principle (OCP)

The Payment class can be extended with new payment methods without modifying the existing payment classes.

3. Liskov Substitution Principle (LSP)

UpiPayment, CardPayment and CashPayment objects can be used wherever a Payment object is expected.

4. Dependency Inversion Principle (DIP) – Deliberately Not Followed

DIP is deliberately not followed in this project because it is a small academic prototype. Direct creation of payment objects keeps the implementation simple and easy to demonstrate.

10. Testing and Results

The system was tested using different test cases to verify its functionality. Test Case

1: Valid Booking

Expected Result: Booking should be confirmed. Status:

PASS

Test Case 2: Multiple Seat Booking

Expected Result: Multiple seats should be booked. Status:

PASS

Test Case 3: Already Booked Seat

Expected Result: Booking should be rejected.

Status: PASS

Test Case 4: Invalid Seat

Expected Result: Error message should be displayed. Status:

PASS

Test Case 5: UPI Payment

Expected Result: Payment should be successful. Status:

PASS

Test Case 6: Card Payment

Expected Result: Payment should be successful. Status:

PASS

Test Case 7: Cash Payment

Expected Result: Payment should be successful. Status:

PASS

Test Case 8: Booking Cancellation

Expected Result: Selected seats should become available. Status:

PASS

Test Case 9: Failed Payment

Expected Result: Booking should not be confirmed. Status:

PASS

Test Case 10: Seat Release After Failed Payment Expected

Result: Seats should become available again. Status: PASS

11. Conclusion

The Movie Ticket Booking System was successfully implemented using C++. The system provides features such as movie and show selection, seat selection, multiple seat booking, payment processing, ticket generation and booking cancellation.

The project demonstrates important OOP concepts such as encapsulation, abstraction, inheritance and polymorphism. Selected SOLID principles were also applied to make the system modular and easier to maintain.

The system was tested for valid bookings, multiple seat selection, already booked seats, invalid inputs, successful payments, failed payments and cancellation. The test results show that the implemented system works correctly for the required scenarios.